

NCBI Bacterial Gene Downloader **(NBGD):** A pipeline for the download and analysis of bacterial **gene and** **protein** datasets

Author: Melissa Maria Pujals Alvarez

Tutor: Eloi Parladé Molist

Typology of work: Data Analysis

Degree: Microbiology (2022 – 2026)

Date of delivery: 25/05/2026

Abstract

The exponential growth of biological sequence data in public databases such as NCBI's has made manual sequence retrieval increasingly impractical. This work presents the NCBI Bacterial Gene Downloader (NBGD), an open-source Python pipeline that automates the retrieval and phylogenetic analysis of bacterial gene and protein sequences from RefSeq, aimed at users with little to no computational background. It consists of two sequential steps: sequence retrieval from Ref-Seq based on user-defined parameters, followed by multiple sequence alignment, trimming and phylogenetic tree inference using third-party tools. Setup scripts and an interactive runner further simplify execution across operating systems. Validation using 16S rRNA sequences across the *Staphylococcus* genus confirmed that the pipeline produces biologically consistent results that are coherent with currently published literature, demonstrating its applicability to real-life cases and research.

Keywords: NCBI, RefSeq, Entrez, Python, Biopython, Command Line Interface (CLI), Tree inference, Bacterial phylogeny, 16S rRNA

TABLE OF CONTENTS

Introduction and Framework.....	4
National Center for Biotechnology Information.....	4
Entrez programming utilities (E-Utilities)	5
Python programming and Biopython	6
Motivation and significance	7
Materials and methods.....	8
Step one – download.py.....	9
Step two – aln_trim_tree.py.....	12
Setup files – run.py and setup.ps1/setup.sh	14
Additional files	16
Results	16
Test case – 16S ribosomal RNA.....	17
Discussion	19
Limitations	20
Future improvements	21
<i>Automated Tree Visualization</i>	21
<i>Sequence Clustering</i>	21
<i>Graphical User Interface (GUI)</i>	21
<i>Interactive Sequence Filtering</i>	22
<i>Customizable FASTA Headers</i>	22
Conclusion	22
Acknowledgments.....	23
Bibliography	23

INTRODUCTION AND FRAMEWORK

NATIONAL CENTER FOR BIOTECHNOLOGY INFORMATION

The National Center for Biotechnology Information (NCBI) (<https://www.ncbi.nlm.nih.gov/>) was established in 1988 as a division of the National Library of Medicine (NLM) at the National Institutes of Health (NIH). It is a government-funded research facility located in the United States¹ that provides a large suite of publicly accessible databases covering a wide range of topics ^{2,3} including (but not limited to) literature (*PubMed Central (PMC), Bookshelf and PubReader*), genes (*Nucleotide, Gene, Genbank, RefSeq*, among others) or proteins (*Protein, the Conserved Domain Database (CDD), COBALT, Protein Clusters*, among others). It is worth noting that these databases are not updated at fixed time intervals, meaning that their contents are in constant flux⁴.

Among these, the Nucleotide database stands out as one of the most comprehensive, serving as a consolidated repository of sequences that brings together data from several sources, including GenBank, RefSeq, the Third Party Annotation database (TPA) and the Protein Data Bank (PDB). GenBank, the primary nucleotide sequence archive², had by release 271.0 (April 2026) accumulated 53.90 trillion bases and 6.27 billion sequence records⁵. Along the same lines, RefSeq ⁶, a curated repository of non-redundant biological sequences and reference genomes, transcripts and proteins, contained as of March 2026 more than 6 trillion bases and almost 600 million records⁶. These figures reflect how advances in sequencing technology have caused the volume of data produced by scientists to grow by orders of magnitude over the last few years, placing increased demand of NCBI databases for both storage and retrieval.

This scale becomes even more remarkable when considering prokaryotic genomes specifically. The number of bacterial sequences available in public databases has grown exponentially over the past decade, and currently RefSeq

alone contains more than 50 million genomic accession corresponding to bacterial genomes⁶.

Given the vast amount of information it offers, NCBI has become an invaluable resource for scientists worldwide. However, to manage this complexity, some specific tools need to be available to researchers. NCBI itself developed its own integrated database retrieval system called *Entrez*^{2,7}, which serves as its primary text-based search and retrieval system. It integrates both literature and molecular databases under a single search interface and offers tools for building precise searches and managing results, such as preset facet filters to narrow down specific results and an Advanced Search interface³ that supports simple Boolean operators (such as AND, OR, or NOT) to generate more targeted and accurate searches.

ENTREZ PROGRAMMING UTILITIES (E-UTILITIES)

To interact programmatically with Entrez, NCBI provides a set of tools known as the *Entrez Programming Utilities* (E-Utilities), that constitute the Application Programming Interface (API). The API includes nine programs that support a uniform set of parameters used to search, link and download data from the 39 Entrez databases⁸:

Table 1. Programs included in the E-utilities

<i>EInfo</i>	Provides the number of records indexed in each field of a given database, the date of the last update of the database, and the available links to other Entrez databases.
<i>ESearch</i>	Responds to a text query with the list of matching unique identifiers (UIDs) in a given database (mainly for later use in ESummary, EFetch or ELink), along with the term translations of the query.
<i>EPost</i>	Accepts a list of UIDs from a given database, stores the set on the History Server, and responds with a query key and web environment for the uploaded dataset.

<i>ESummary</i>	Responds to a list of UIDs from a given database with the corresponding document summaries.
<i>EFetch</i>	Responds to a list of UIDs in a given database with the corresponding data records in a specified format. EFetch is currently supported in the following databases: PubMed, PubMed Central, Journals, Nucleotide, Protein, Genome, Gene, SNP, PopSet, and Taxonomy.
<i>ELink</i>	Responds to a list of UIDs in a given database with either a list of related UIDs (and relevancy scores) in the same database or a list of linked UIDs in another Entrez database.
<i>EGQuery</i>	Responds to a text query with the number of records matching the query in each Entrez database.
<i>ESpell</i>	Retrieves spelling suggestions for a text query in a given database.
<i>ECitMatch</i>	Retrieves PubMed IDs (PMIDs) corresponding to a set of input citation strings.

Although the majority of the data at NCBI is indexed in Entrez, the retrieval engine of this system is only capable of retrieving data that is already present in Entrez, which should be taken into account when selecting the data sources for a given analysis.

These server-side programs were designed to enable the automation of data extraction, allowing users to bypass the web interface when needed, for example, to pull large datasets, perform recurring queries, or integrate NCBI data into external software pipelines.

PYTHON PROGRAMMING AND BIOPYTHON

Python is an interpreted, object-oriented, high-level programming language with dynamic semantics created in the late 1980's by Guido van Rossum. Its simple, easy to read syntax emphasizes readability and therefore reduces the cost of program maintenance⁹.

For these reasons, it is considered one of the most popular programming languages in use today, and one of its most appealing features is the availability of a large collection of extension libraries contributed by its user community. These characteristics make Python a particularly suitable choice for building accessible bioinformatics tools, since it has a clear and readable syntax and an extensive library of modules that provide all the necessary functionality to interact with NCBI databases without the need for complex code.

Among these libraries, Biopython¹⁰ stands out by being a comprehensive selection of modules intended for computational biology or bioinformatics researchers to use either in standalone scripts or as a part of larger software projects. It is a widely utilized tool among researchers, given it preserves the readability and extensibility of Python while also incorporating several scientific computing packages such as *Bio.SeqIO*^{10,11} to input and output assorted sequence file formats, or *Bio.Cluster*^{10,12,13}, which provides commonly used clustering algorithms.

Among these modules, one that stands out is *Bio.Entrez*^{10,14}, which provides programmatic access to *Entrez* by making use of the aforementioned E-utilities, allowing users to query and retrieve data from NCBI databases directly from their scripts.

MOTIVATION AND SIGNIFICANCE

While the *Entrez* web portal allows any user with a standard browser to retrieve sequences from NCBI, this approach becomes impractical as the scale of the task grows, considering that the enormous volume of data now available at these databases makes manual approaches unfeasible for meaningful analyses. Furthermore, repetitive manual downloads are not only time consuming but also error prone, which can compromise the reproducibility of scientific work. This lack of reproducibility is a recognized problem in bioinformatics.

The E-utilities API and libraries like Biopython, described in the preceding sections, offer a powerful alternative that lets the user bypass the web interface and build a more robust and efficient pipeline for data retrieval and analysis.

However, they require programming skills and their use may be challenging for inexperienced users.

The ability to retrieve specific genes from a defined taxonomic group in an automated and reproducible manner has direct applications in several areas of biological research. One particularly relevant example is the study of antimicrobial resistance (AMR), where identifying and comparing the distribution of resistance genes across bacterial taxa is a key step in understanding how resistance spreads and evolves. Similarly, the targeted retrieval of 16S ribosomal RNA sequences has been historically a key tool for defining and classifying bacterial species. Both of these use cases require the ability to query and download large, curated sets of sequences in a reproducible and automated manner.

Based on everything stated thus far, the aim of this work is to create a ready-to-use pipeline for the download of gene and protein sequences of bacterial origin available at the RefSeq database from NCBI in standard formats suited for further analysis. It is aimed at users with little to no computational background, who can run it through simple command-line instructions.

Additionally, a common limitation of existing bioinformatics tools is the lack of accessible information aimed at a beginner audience, and for that, this pipeline will have an accompanying documentation file and a dedicated webpage that will help not only to understand how to use the pipeline, but potentially learn new concepts of coding and bioinformatics.

MATERIALS AND METHODS

To achieve the objective, the pipeline was divided in two sequential steps: the first retrieves nucleotide or protein sequences from NCBI RefSeq database based on user-defined parameters, and the second performs multiple sequence alignment, trimming, and phylogenetic tree inference on the downloaded dataset. Additional third-party tools are invoked at the second stage, as detailed in the following sections.

Both steps were implemented in python 3 (Python Software Foundation; <https://www.python.org>), using the Biopython library for programmatic access to NCBI's utilities and sequence handling, Openpyxl¹⁵ for excel file writing, Certifi¹⁶ for validating the trustworthiness of SSL/TLS certificates when accessing NCBI, and Clipkit for multiple sequence alignment-trimming.

The pipeline is executed via a command-line interface (CLI) and is compatible with any operative system (OS) on which python 3 is available. Every analysis provided in this text was performed on python version 3.12.10, but has also been successfully tested on version 3.10 and 3.13.

STEP ONE – DOWNLOAD.PY

This first script, *download.py*, automates the retrieval of gene or protein sequences from NCBI's RefSeq database. As shown in Figure 1, the program queries the RefSeq assembly database for a specified organism or Taxonomic Identifier (TaxID), follows the links to the underlying nucleotide records in the nuccore database (which includes GenBank and RefSeq), scans the CDS feature annotations for matches to the requested gene or product, and extracts either the CDS nucleotide sequences or their translated protein, depending on the user's choice. The script interacts with NCBI programmatically through Biopython's Entrez module.

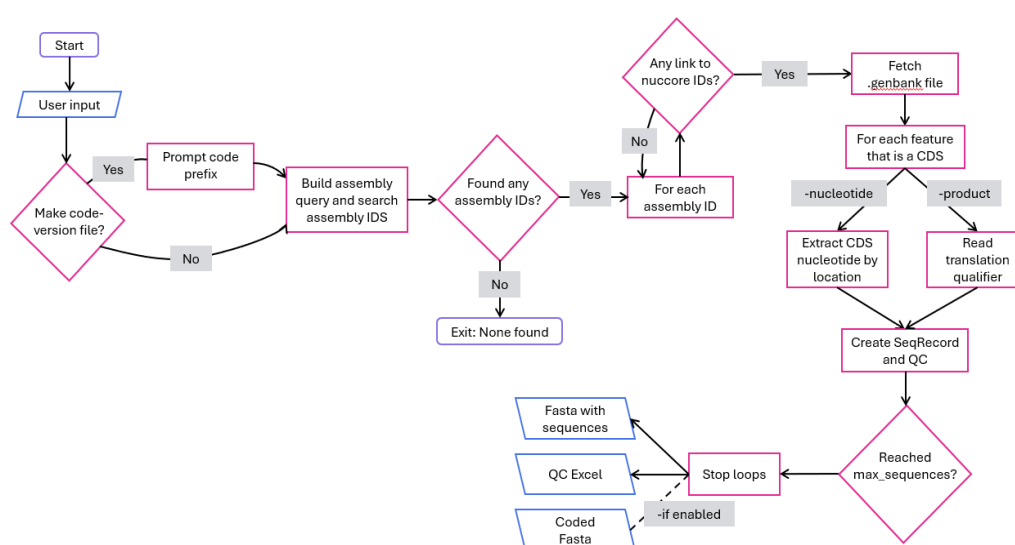


Figure 1. Workflow of *download.py*

To go in depth about the behavior of the script, upon execution it prompts the user for the following input parameters (Figure 2):

- NCBI registered e-mail address, required by NCBI's Entrez utilities for all programmatic access.
- NCBI API key, optional but strongly recommended to increase the request rate limit for large downloads.
- Sequence type, specifying whether it's nucleotide or amino-acid sequences to be retrieved.
- Gene or product names(s) to be used as match terms.
- TaxID or organism name, used to restrict the search to a specific taxonomic group. Given the structure of the pipeline, this parameter is highly recommended because the program is specifically designed for bacterial, archaeal or viral organisms. Its use with eukaryotic genomes is not recommended, as the complexity and size of eukaryotic assemblies fall outside the scope of the pipeline's design.
- Maximum RefSeq assemblies to scan and maximum number of sequences to retrieve.
- Output filename, which default to "download" if not specified.
- Whether to create an additional FASTA file with user-defined sequence identifiers, intended to simplify downstream analyses.

Using the provided parameters, the program builds an Entrez search query targeting the most recent RefSeq assemblies associated with the specified TaxID/organism.

For each assembly retrieved, the program attempts to identify the linked nuccore records. Assemblies for which no nuccore link is found are skipped, but for those that resolve successfully, the corresponding GenBank file is fetched and its CDS feature annotations are parsed. The program then attempts to match the user-provided query to the following GenBank qualifiers: /gene, /gene_synonym, /product, /note. Only exact matches are considered.

```

=== Extract Gene Products by Annotation ===
This extracts CDS by gene/product annotation from RefSeq genome records in NCBI.

Email for NCBI Entrez (required by NCBI): MelissaMaria.Pujals@autonoma.cat
NCBI API key (optional, recommended): 6ee173a05fd6056fa97d8f440b85138f9208
Search for (nucleotide | protein) sequence: nucleotide
Gene name(s) (comma-separated): mecA, MecA
TaxID (optional, recommended): 1280
Organism name (recommended if no TaxID):
Max RefSeq assemblies to scan: 200
Max extracted sequences to output (optional):
Output filename base [download]: meca
Create additional FASTA file with a numbered code? [Y/N]: y
Code name (e.g., SAU): MEC

Assembly query (RefSeq-only):
(latest_refseq[filter]) AND (txid1280[Organism:exp])

Scanned assemblies: 23/200 | extracted: 28

```

Figure 2. Screenshot of NBGD running under a Windows system, downloading sequences of *mecA* gene searched in 200 *Staphylococcus aureus* assemblies (TaxID: 1280). The inputs required can be seen.

Upon a successful match, the corresponding nucleotide or protein sequence is extracted and stored as a Biopython SeqRecord object. This process iterates until either the maximum number of assemblies has been scanned or the maximum number of sequences has been reached, at which point the loop terminates.

The program then writes the retrieved sequences on a FASTA file and, if enabled, an additional FASTA file in which sequence identifiers are replaced by an alphabetical code defined by the user. In addition, a Microsoft Excel file is generated containing all relevant information and metadata for each retrieved sequence (including accession numbers, organism names, annotation qualifiers and, if enabled, the corresponding code for each sequence), to allow the user to perform a quality control (QC) review of the results. A brief QC summary is also printed in the terminal upon the search completion.

The outputs generated by a representative run of *download.py* (Figure 2) are shown in Figure 3.

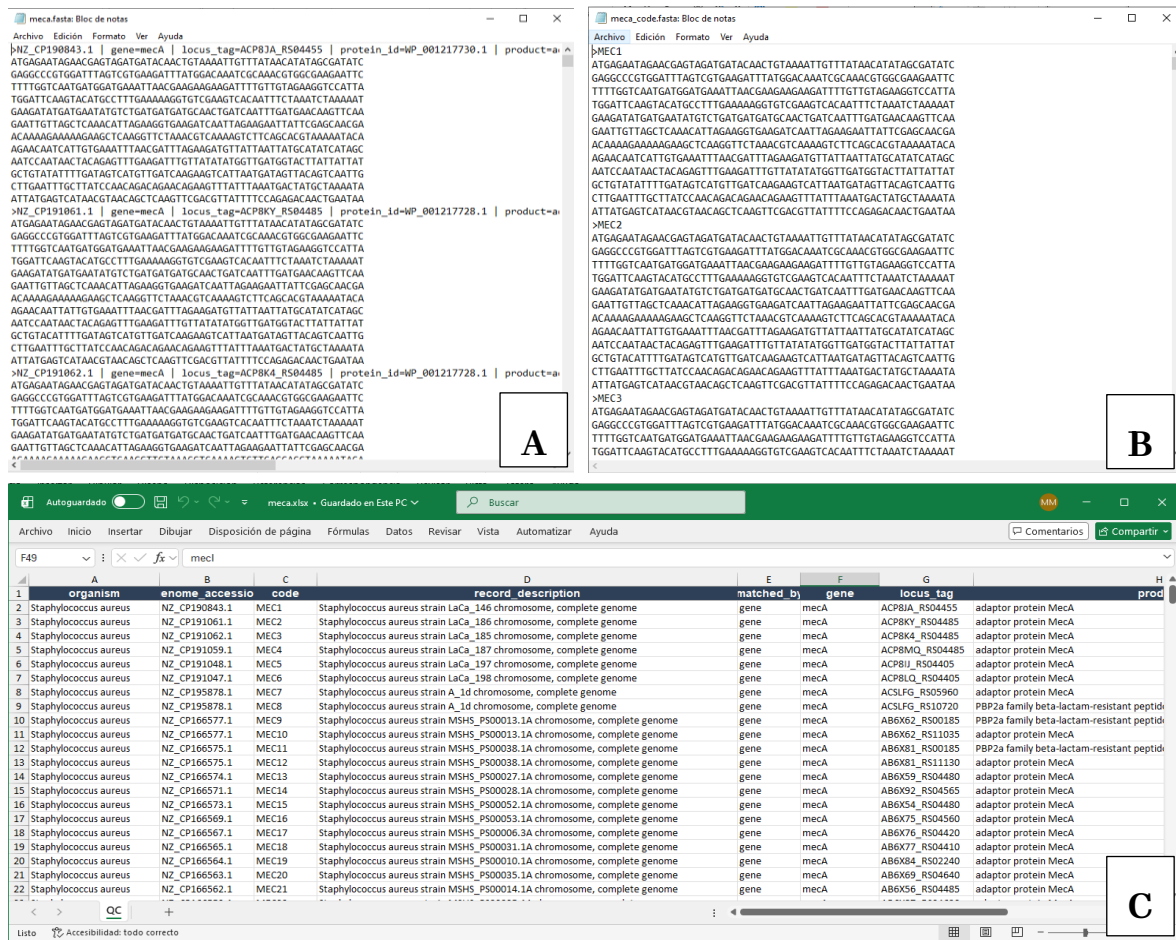


Figure 3. Outputs generated by the representative run of `download.py`. A: FASTA file with the nucleotide sequences fetched. B: Additional FASTA file with the user-provided alphabetical code. C: QC Excel file with the necessary QC information.

STEP TWO – `aln_trim_tree.py`

Once step one is complete and the sequences have been retrieved, the second script, `aln_trim_tree.py`, processes and analyzes the data through a sequential three-stage pipeline (Figure 4): multiple sequence alignment using MUSCLE¹⁷, alignment trimming using Clipkit¹⁸ and phylogenetic tree inference using IQTREE¹⁹.

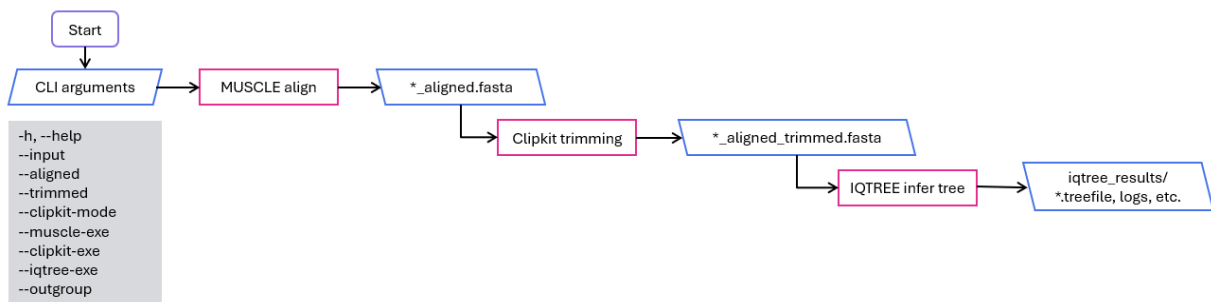


Figure 4. Workflow of `aln_trim_tree.py`.

A distinguishing feature of this step is the use of third-party command-line tools, namely MUSCLE and IQTREE, which introduces a platform-dependent installation procedure. On Windows systems, the user must download the executable binaries for each tool and either add them to the system PATH or place them directly in the script's working directory so that they can be located when running. On Unix-based and macOS systems, the tools can be installed via the system package manager using CLI such as “*sudo apt install*” or “*brew install*”, respectively.

These tools were selected on the basis of their widespread adoption in the phylogenetic literature, their availability as standalone command-line executables compatible with cross-platform deployment, and their open-source licensing.

Provided that the tools are correctly installed and accessible, the script performs a sequential analysis of the input data. The script is executed via the command-line interface and accepts a range of optional and required arguments. A full description of these can be accessed by running the script with the -h flag (Figure 5).

The only mandatory input is the sequence file to be analyzed. Additionally, the user may specify custom output filenames for the alignment and trimmed alignment files, define a custom path to the tools executable (applicable to Windows systems only), and select tool-specific parameters that include the Clipkit trimming mode (which defaults to “smart-gap”, as it is the most broadly applicable option across different alignment types) and the outgroup sequence for phylogenetic tree rooting (which defaults to the first sequence in the alignment if not specified).

```

C:\Users\Uni\Desktop\Bachelor_thesis_Saureus\2 ALN_TRIM_TREE>py aln_trim_tree.py -h
usage: aln_trim_tree.py [-h] --input INPUT [--aligned ALIGNED] [--trimmed TRIMMED] [--clipkit-mode CLIPKIT_MODE]
                        [--muscle-exe MUSCLE_EXE] [--clipkit-exe CLIPKIT_EXE] [--iqtree-exe IQTREE_EXE]
                        [--outgroup OUTGROUP]

Align (MUSCLE) -> Trim (ClipKIT) -> Tree (IQ-TREE) [Cross-platform].

options:
  -h, --help            show this help message and exit
  --input INPUT          Input FASTA file (unaligned sequences)
  --aligned ALIGNED      Output FASTA file for alignment (default: <input>_aligned.fasta)
  --trimmed TRIMMED      Output FASTA file for trimmed alignment (default: <input>_aligned_trimmed.fasta)
  --clipkit-mode CLIPKIT_MODE
                        ClipKIT trimming mode (default: smart-gap)
  --muscle-exe MUSCLE_EXE
                        Path to MUSCLE executable, or leave blank to auto-discover
  --clipkit-exe CLIPKIT_EXE
                        Path to ClipKIT executable (default: clipkit in current Python)
  --iqtree-exe IQTREE_EXE
                        Path to IQ-TREE executable, or leave blank to auto-discover
  --outgroup OUTGROUP    Outgroup name for IQ-TREE (optional)

C:\Users\Uni\Desktop\Bachelor_thesis_Saureus\2 ALN_TRIM_TREE>

```

Figure 5. CLI arguments help for *aln_trim_tree.py*.

For phylogenetic tree inference, IQTREE runs by default in ModelFinder model^{19,20}, a built-in model selection procedure that automatically evaluates a large subset of models to determine the one that best fits the sequence alignment.

SETUP FILES – RUN.PY AND SETUP.PS1/SETUP.SH

To improve accessibility and make a more user-friendly pipeline, two auxiliary files were developed alongside the main scripts.

The first, *run.py*, serves as an interactive pipeline runner. As shown in Figure 6, upon execution the script prints a menu prompting the user to select which steps of the pipeline they wish to perform, and subsequently runs the corresponding script.

```

=== Pipeline runner ===
1) Step 1 (download/extract sequences)
2) Step 2 (align/trim/tree)
3) Step 1 + Step 2
Choose an option [1/2/3]:

```

Figure 6. CLI menu of *run.py*.

Furthermore, *run.py* is adapted to handle the command-line arguments associated with step 2 (Figure 7). When step 2 is selected (and when it's run sequentially as well), the script prompts the user to specify the relevant arguments, while also accepting the *-h* flag in case the user wishes to consult all the available options.

```
Name of the input FASTA for Step 2: meca_code.fasta
(Optional) Advanced Step 2 arguments:
Examples:
  --clipkit-mode X --outgroup X -muscle-exe /path/to/muscle
  Leave blank to use defaults.
  (Tip: type -h to see all Step 2 options.)
Extra args:

Running: ['C:\\Users\\Un1\\Desktop\\NBGD\\MEC_A\\.venv\\Scripts\\python.exe', '2 ALN_TRIM_TREE\\aln_trim_tree.py', '--input', 'meca_code.fasta']
Running: C:\\Users\\Un1\\Desktop\\NBGD\\MEC_A\\2 ALN_TRIM_TREE\\muscle-win64.v5.3.exe -align meca_code.fasta -output meca_code_aligned.fasta

muscle 5.3.win64 [d9725ac] 17.1Gb RAM, 12 cores
Built Nov 10 2024 22:59:05
(C) Copyright 2004-2021 Robert C. Edgar.
https://drive5.com

[align meca_code.fasta]
Input: 330 Seqs, avg length 1119, max 2010, min 345

00:00 1.8Mb 100.0% Derep 36 uniques, 294 dupes
00:00 1.9Mb CPU has 12 cores, running 12 threads
00:19 36Mb 100.0% Calc posteriors
00:19 36Mb 100.0% UPGMA5
00:26 71Mb 100.0% Consistency (1/2)
```

Figure 7. Screenshot of *run.py* running specifically Step 2 of the pipeline, with the extra argument of a specific outgroup for IQTREE.

The second file is a platform-dependent setup script available in two versions: *setup.ps1* for Windows systems and *setup.sh* for Unix/macOS systems. Both scripts automate the initial configuration of the program by creating a Python virtual environment, activating it, installing all the required Python libraries and dependencies and launching the aforementioned *run.py*, enabling the user to initialize and run the full pipeline with a single command.

The use of a virtual environment ensures that the required packages are installed in an isolated context, which avoids conflicts with globally installed packages and keeps the system environment unaffected.

Additionally, *setup.sh* extends this functionality by automatically installing the required third-party tools stated in the previous section. It does so via the system package manager, a step that on Windows must be performed manually.

ADDITIONAL FILES

The remaining files in the root directory serve auxiliary roles in support of the pipeline. The *requirements.txt* contains a list of all the necessary Python libraries and their respective versions (if needed), and is referenced by both *setup.ps1* and *setup.sh* during the automated installation of dependencies.

Also included is the *README.md* file, which provides a general overview of the project and detailed usage instructions for all components of the pipeline. Complementing this, the *command.txt* file contains the exact terminal command required to initiate the pipeline, allowing the user to simply copy and paste it without needing to construct the command manually.

Finally, the *docs* folder contains files pertaining exclusively to the project's associated webpage and is not intended to be accessed or modified by the user.

RESULTS

The primary outcome of this project is the development of a fully-functional and open-source bioinformatics pipeline for the download of gene and gene product sequences from NCBI's RefSeq database.

The pipeline, comprising all scripts, setup and auxiliary files, and documentation described in the Materials and Methods section is publicly available at <https://github.com/pujals298/NBGD-NCBI-Bacterial-Gene-Downloader>, and is accompanied by a dedicated documentation webpage at <https://pujals298.github.io/NBGD-NCBI-Bacterial-Gene-Downloader/>, which provides extensive information not only about the pipeline itself, but also bioinformatics key concepts and inquiries.

To demonstrate the functionality of the pipeline and validate its outputs, a representative analysis was performed using the 16S ribosomal RNA gene across the *Staphylococcus* genus (TaxID: 1279) as a test case. The 16S rRNA gene was selected given its widespread use in prokaryotic taxonomy as a phylogenetic marker and the extensive literature regarding the *Staphylococcus* genus, against which the pipeline's outputs could be compared and assessed.

TEST CASE – 16S RIBOSOMAL RNA

The search was conducted across 200 RefSeq Staphylococcus genus assemblies (Figure 8), with the maximum number of output sequences capped at 400 to ensure a dataset of sufficient size for phylogenetic inference while avoiding redundancy and excessive computational cost in the downstream steps.

```
=== Extract Gene Products by Annotation ===  
  
This extracts CDS by gene/product annotation from RefSeq genome records in NCBI.  
  
Email for NCBI Entrez (required by NCBI): MelissaMaria.Pujals@autonoma.cat  
NCBI API key (optional, recommended): 6ee173a05fd6056fa97d8f440b85138f9208  
Search for (nucleotide | protein) sequence: nucleotide  
Gene name(s) (comma-separated): 16S ribosomal RNA  
TaxID (optional, recommended): 1279  
Organism name (recommended if no TaxID):  
Max RefSeq assemblies to scan: 200  
Max extracted sequences to output (optional): 400  
Output filename base [download]: rrna  
Create additional FASTA file with a numbered code? [Y/N]: y  
Code name (e.g., SAU): RNA  
  
Assembly query (RefSeq-only):  
(latest_refseq[filter]) AND (txid1279[Organism:exp])  
  
Scanned assemblies: 67/200 | extracted: 396  
  
=== Run summary (QC) ===  
Assemblies scanned: 68  
Assemblies with ≥1 hit: 68  
GenBank records parsed: 182  
CDS features scanned: 175461  
CDS matches found: 400  
Sequences extracted (written to FASTA): 400  
  
Done. Extracted 400 sequences from 68 RefSeq assemblies.  
FASTA: C:\Users\Uni\Desktop\NBGD\rrna.fasta  
Code FASTA: C:\Users\Uni\Desktop\NBGD\rrna_code.fasta  
Excel: C:\Users\Uni\Desktop\NBGD\rrna.xlsx  
  
Done.
```

Figure 8. Screenshot of a NBGD representative run. It has fetched 400 16S ribosomal RNA sequences in 68 assemblies of Staphylococcus genus (TaxID: 1279). The inputs required and summary QC can be seen.

The QC summary shown in Figure 8 confirms that the sequences were successfully retrieved and the output files correctly generated in the first step of the pipeline. It can be seen that, with the specified parameters, the script yielded a total of 400 16S rRNA sequences distributed across 8 distinct Staphylococcus species, as recorded in the QC Excel file generated (Figure 9).

Autoguardado



rrna.xlsx · Guardado en Este PC

 Buscar

ArchivoInicioInsertarDibujarDisposición de páginaFórmulasDatosRevisarVistaAutomatizarAyuda

A2

Staphylococcus capitis

	A	B	C	D	E	F	G	H
	organism	genome_access	code	record_description	matched	gene	locus_tag	product
1	Ordenar de A a Z	77.1	RNA1	Staphylococcus capitis strain LaCa_95 chromosome, complete genon product		ACP815_RS03565	16S ribosomal RNA	
2	Ordenar de Z a a	77.1	RNA2	Staphylococcus capitis strain LaCa_95 chromosome, complete genon product		ACP815_RS04080	16S ribosomal RNA	
3	Ordenar por color	77.1	RNA3	Staphylococcus capitis strain LaCa_95 chromosome, complete genon product		ACP815_RS04695	16S ribosomal RNA	
4	Vista de Hoja	77.1	RNA4	Staphylococcus capitis strain LaCa_95 chromosome, complete genon product		ACP815_RS10720	16S ribosomal RNA	
5	Borrar filtro de "organism"	77.1	RNA5	Staphylococcus capitis strain LaCa_95 chromosome, complete genon product		ACP815_RS10735	16S ribosomal RNA	
6	Filtrar por color	77.1	RNA6	Staphylococcus capitis strain LaCa_95 chromosome, complete genon product		ACP815_RS10985	16S ribosomal RNA	
7	Filtros de texto	77.1	RNA7	Staphylococcus capitis strain LaCa_96 chromosome, complete genon product		ACP8KJ_RS03570	16S ribosomal RNA	
8	Buscar	77.1	RNA8	Staphylococcus capitis strain LaCa_96 chromosome, complete genon product		ACP8KJ_RS04085	16S ribosomal RNA	
9		77.1	RNA9	Staphylococcus capitis strain LaCa_96 chromosome, complete genon product		ACP8KJ_RS04700	16S ribosomal RNA	
10		77.1	RNA10	Staphylococcus capitis strain LaCa_96 chromosome, complete genon product		ACP8KJ_RS10770	16S ribosomal RNA	
11		77.1	RNA11	Staphylococcus capitis strain LaCa_96 chromosome, complete genon product		ACP8KJ_RS10785	16S ribosomal RNA	
12		77.1	RNA12	Staphylococcus capitis strain LaCa_96 chromosome, complete genon product		ACP8KJ_RS11035	16S ribosomal RNA	
13		34.1	RNA13	Staphylococcus hominis strain LaCa_103 chromosome, complete ger product		ACP8K8_RS03345	16S ribosomal RNA	
14		34.1	RNA14	Staphylococcus hominis strain LaCa_103 chromosome, complete ger product		ACP8K8_RS03835	16S ribosomal RNA	
15		34.1	RNA15	Staphylococcus hominis strain LaCa_103 chromosome, complete ger product		ACP8K8_RS04370	16S ribosomal RNA	
16		34.1	RNA16	Staphylococcus hominis strain LaCa_103 chromosome, complete ger product		ACP8K8_RS10340	16S ribosomal RNA	
17		34.1	RNA17	Staphylococcus hominis strain LaCa_103 chromosome, complete ger product		ACP8K8_RS10355	16S ribosomal RNA	
18		34.1	RNA18	Staphylococcus hominis strain LaCa_103 chromosome, complete ger product		ACP8K8_RS10600	16S ribosomal RNA	
19		56.1	RNA19	Staphylococcus epidermidis strain LaCa_109 chromosome, complete product		ACP8MF_RS03820	16S ribosomal RNA	
20		56.1	RNA20	Staphylococcus epidermidis strain LaCa_109 chromosome, complete product		ACP8MF_RS04300	16S ribosomal RNA	
21		56.1	RNA21	Staphylococcus epidermidis strain LaCa_109 chromosome, complete product		ACP8MF_RS04905	16S ribosomal RNA	
22		56.1	RNA22	Staphylococcus epidermidis strain LaCa_109 chromosome, complete product		ACP8MF_RS11230	16S ribosomal RNA	
23		56.1	RNA23	Staphylococcus epidermidis strain LaCa_109 chromosome, complete product		ACP8MF_RS11245	16S ribosomal RNA	
24		56.1	RNA24	Staphylococcus epidermidis strain LaCa_109 chromosome, complete product		ACP8MF_RS11495	16S ribosomal RNA	

Ordenar de A a Z

Ordenar de Z a a

Ordenar por color

Vista de Hoja

Borrar filtro de "organism"

Filtrar por color

Filtros de texto

Buscar

(Seleccionar todo)

☒ Staphylococcus aureus

☒ Staphylococcus capitis

☒ Staphylococcus epidermidis

☒ Staphylococcus haemolyticus

☒ Staphylococcus hominis

☒ Staphylococcus lugdunensis

☒ Staphylococcus pettenkoferi

☒ Staphylococcus warneri

ACEPTAR

Cancelar

Figure 10. QC Excel file showing that the 16S rRNA of 8 different species of *Staphylococcus* (pink box) were retrieved.

The coded version of the sequences (also generated in step 1) were subsequently processed through the second script of the pipeline, *aln_trim_tree.py*, which correctly performed the multiple sequence alignment, trimming and phylogenetic tree inference with the previously mentioned third-party tools. The resulting tree file (.treefile, in standard Newick format), was visualized using the Interactive Tree of Life web platform (iTOL; <https://itol.embl.de/>), an external tool commonly used for the rendering and annotation of phylogenetic trees (Figure 10).

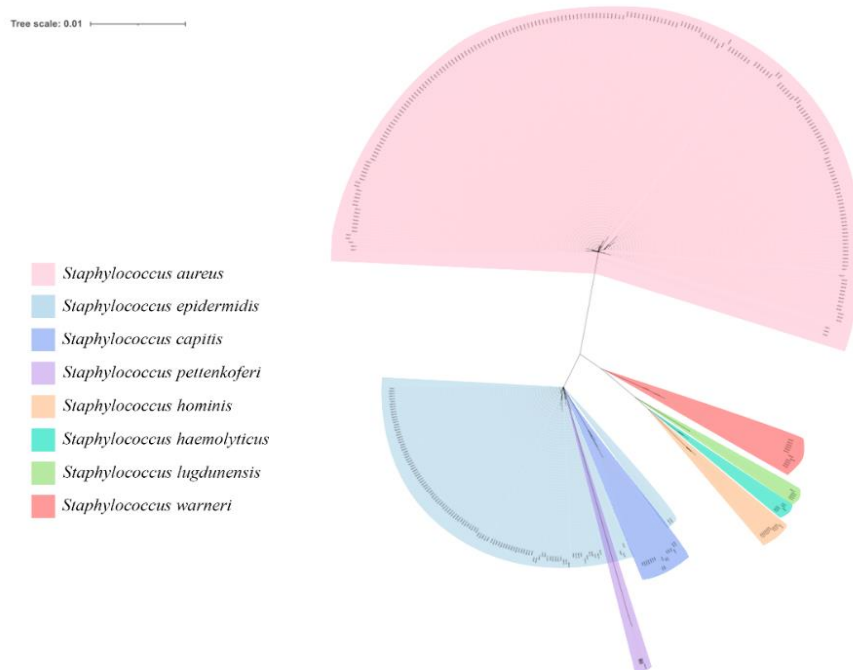


Figure 9. Phylogenetic tree inferred from the retrieved 16S rRNA sequences of the *Staphylococcus* genus. The 8 differentiated clades are color-coded. Made with iTOL.

As seen in Figure 10, the phylogenetic tree inferred from the retrieved 16S rRNA sequences successfully distributed all 8 *Staphylococcus* species (*S. aureus*, *S. epidermidis*, *S. capitis*, *S. pettenkoferi*, *S. hominis*, *S. haemolyticus*, *S. lugdunensis* and *S. warneri*) in distinct clades, consistent with previous published phylogenies of the genus²¹⁻²⁴. The relative clade sizes reflect the unequal representation of species in RefSeq, with *S. aureus* and *S. epidermidis* being the largest clades, as expected given their clinical relevance and, consequently, higher sequencing prevalence in public databases.

These results demonstrate that the pipeline is capable of retrieving, aligning and inferring a biologically coherent phylogeny from RefSeq genomic data in an automated and reproducible manner.

DISCUSSION

The pipeline presented in this work provides an automated solution for the retrieval and phylogenetic analysis of prokaryotic and viral gene and protein sequences, and the validation test case demonstrated that it is capable of producing biologically meaningful results. All of the previously described scripts address a practical need in microbial genomics for accessible and reproducible sequence retrieval.

Nevertheless, several existing tools address aspects of the workflow implemented in this pipeline. NCBI's own Datasets command-line tool and the Entrez E-utilities provide programmatic access to NCBI databases, but those require the user to handle the downstream processing, alignment and tree inference independently. Other platforms such as MEGA or many tools at EMBL-EBI offer alignment and tree inference capabilities, but rely on manual sequence input and do not integrate any sequence-retrieval system. The pipeline described throughout the work intends to fill a practical need in the bioinformatics landscape, while remaining accessible to all kinds of users.

LIMITATIONS

Despite its functionality, the pipeline presents some limitations that are important to acknowledge. Some arise from design decisions, while others reflect technical challenges encountered during the development that could not be fully resolved within the scope of this work.

The most significant one concerns the inconsistency of gene and product annotations across RefSeq records. GenBank annotations are deposited by a large number of independent researchers and automated pipelines, and as a result the same gene or product may be annotated under different names or qualifiers across different genome records. Since the pipeline relies on matching to these annotations to identify and extract sequences of interest, its performance is inherently dependent on the consistency of the annotations for said gene.

A related limitation is the redundancy of current online databases. Since multiple genome assemblies may be deposited for the same strain or isolate, the retrieved dataset may contain highly similar or near-identical sequences, particularly when scanning a large number of assemblies. While this issue is partially remediated by using a non-redundant database such as RefSeq and the generation of the Excel QC file so as the user can manually address it, no automated deduplication step is currently implemented in the pipeline.

From a design perspective, the pipeline is explicitly intended for use with bacterial, archaeal and/or viral organisms, and is not suited for eukaryotic genomes, whose size and structural complexity fall outside the intended range of the project.

Additionally, as the pipeline depends on real time access to NCBI's servers via the Entrez API, its performance is subject to network availability and server responsiveness, which can occasionally result in slower downloads or connection errors, particularly for large datasets.

FUTURE IMPROVEMENTS

Although the pipeline in its current form fulfils its intended purpose, a number of improvements have been identified that would significantly broaden its functionality and usability, some of which are described next.

Automated Tree Visualization

As noted in the Results section, the pipeline currently produces a phylogenetic tree in the *Newick* format, the visualization of which requires an external tool. This might not be ideal for some users who wished to perform every step locally, so an additional script capable of automatically rendering the tree into a adequate figure would be a natural extension of the pipeline. This could be implemented in Python using libraries such as ETE3²⁵, or in RStudio using packages such as ggtre^{26,27} or even msaplot²⁶, both of which offer extensive tree annotation and visualization tools.

Sequence Clustering

For analysis that involve large datasets, the retrieved sequences may contain a high degree of redundancy, with many nearly identical sequences that contribute little phylogenetic information while substantially increasing the computational cost. The integration of a sequence clustering step using tools such as MMseqs2²⁸ or CD-HIT^{29,30} prior to the alignment would allow the user to reduce the dataset size by maintaining only representative sequences, improving both efficiency and clarity for downstream phylogenetic analysis.

Graphical User Interface (GUI)

The current version relies entirely on a command-line interface, which, despite the interactive prompts and setup scripts designed to ease the user experience, may still present difficulties for users with no prior programming or terminal experience. The development of a Graphical User Interface (GUI) would make the pipeline accessible to a considerably broader audience.

Interactive Sequence Filtering

At the moment, the user has no option to review, select and automatically remove sequences between the retrieval and alignment steps, only in a manual form. The addition of an interactive filtering step in which the user is presented with the retrieved sequences and given the option to exclude specific entries based on criteria such as sequence length, organism, or annotation, would provide more control over the quality of the input data for the downstream analysis.

Customizable FASTA Headers

Currently, the sequence identifiers in the output FASTA file default to the genome accession number, with the option of a numerical code defined by the user as an alternative. To allow the user to construct custom headers from a selection of available metadata fields (such as organism name, gene name, locus tag or protein ID) would improve the readability and interpretability of the subsequent analysis and particularly tree visualization.

CONCLUSION

Although querying the NCBI database and downloading the respective sequences can usually be done manually from the web browser, the pipeline presented in this work intends to solve a practical gap detected in prokaryotic bioinformatics, by offering a fully-functional, reproducible and accessible alternative. Despite its limitations, the validation analysis demonstrated that it is capable of producing biologically coherent and interpretable results, and the improvements outlined above would substantially expand the pipeline's applicability and transform it to a more robust research tool.

It is hoped that the pipeline may serve as a useful resource for any user who requires a programmatic and annotation-driven approach to sequence retrieval and phylogenetic inference in microbial genomics.

ACKNOWLEDGMENTS

The scripts described in this work were developed with the aid of GitHub Copilot (Microsoft), with the GPT-5.2 and GPT-4.1 models. All AI-generated code was carefully reviewed and validated throughout the development process, and the scripts have been heavily annotated to ensure that anyone wishing to understand their logic may do so. Everything in this document was written by the author.

BIBLIOGRAPHY

1. Cooper, P. S. *et al.* Education resources of the National Center for Biotechnology Information. *Brief. Bioinform.* **11**, 563 (2010).
2. Agarwala, R. *et al.* Database resources of the National Center for Biotechnology Information. *Nucleic Acids Res.* **44**, D7–D19 (2016).
3. Cooper, P. & Romiti, M. In: Entrez® Help [Internet]. *Bethesda (MD): National Center for Biotechnology Information (US)* <https://www.ncbi.nlm.nih.gov/books/NBK3837/> (2024).
4. Wojtkiewicz, M., Wiederin, J. & Ciborowski, P. Proteomic Database Search and Analytical Quantification for Mass Spectrometry. *Proteomic Profiling and Analytical Chemistry: The Crossroads: Second Edition* 193–210 (2016) doi:10.1016/B978-0-444-63688-1.00011-2.
5. Sayers, E. W. *et al.* GenBank 2025 update. *Nucleic Acids Res.* **53**, D56–D61 (2025).
6. Goldfarb, T. *et al.* NCBI RefSeq: reference sequence standards through 25 years of curation and annotation. *Nucleic Acids Res.* **53**, D243–D257 (2025).
7. Maglott, D., Ostell, J., Pruitt, K. D. & Tatusova, T. Entrez Gene: gene-centered information at NCBI. *Nucleic Acids Res.* **39**, D52 (2010).

8. NCBI Resource Coordinators. Entrez® Programming Utilities Help. *Bethesda (MD): National Center for Biotechnology Information (US)* <https://www.ncbi.nlm.nih.gov/books/NBK25501/> (2010).
9. Python Software Foundation. What is Python? Executive Summary. <https://www.python.org/doc/essays/blurb/>.
10. Cock, P. J. A. *et al.* Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics* **25**, 1422 (2009).
11. Biopython Contributors. SeqIO — Sequence Input/Output. <https://biopython.org/wiki/SeqIO>.
12. de Hoon, M. J. L., Imoto, S., Nolan, J. & Miyano, S. Open source clustering software. *Bioinformatics* **20**, 1453–1454 (2004).
13. Biopython Contributors. Cluster analysis — Biopython Tutorial (v1.88). https://biopython.org/docs/dev/Tutorial/chapter_cluster.html.
14. Biopython Contributors. Bio.Entrez API documentation (v1.76). <https://biopython.org/docs/1.76/api/Bio.Entrez.html>.
15. Gazoni, E. & Clark, C. openpyxl - A Python library to read/write Excel 2010 xlsx/xlsm files (Version 3.1.3). <https://openpyxl.readthedocs.io/en/stable/> (2024).
16. Reitz, K. certifi: Python package for providing Mozilla’s CA Bundle. <https://pypi.org/project/certifi/>.
17. Edgar, R. C. Muscle5: High-accuracy alignment ensembles enable unbiased assessments of sequence homology and phylogeny. *Nature Communications* **2022 13:1** **13**, 6968- (2022).
18. Steenwyk, J. L., Buida, T. J., Li, Y., Shen, X. X. & Rokas, A. ClipKIT: A multiple sequence alignment trimming software for accurate phylogenomic inference. *PLoS Biol.* **18**, e3001007 (2020).
19. Wong, T. K. F. *et al.* IQ-TREE 3: phylogenomic inference software using complex evolutionary models. *Mol. Biol. Evol.* **43**, 1–6 (2026).

20. Kalyaanamoorthy, S., Minh, B. Q., Wong, T. K. F., Von Haeseler, A. & Jermini, L. S. ModelFinder: fast model selection for accurate phylogenetic estimates. *Nature Methods* 2017 14:6 **14**, 587–589 (2017).
21. Kosecka-Strojek, M. *et al.* Development and Validation of a Reference Data Set for Assigning Staphylococcus Species Based on Next-Generation Sequencing of the 16S-23S rRNA Region. *Front. Cell. Infect. Microbiol.* **9**, 464039 (2019).
22. Ghebremedhin, B., Layer, F., König, W. & König, B. Genetic classification and distinguishing of Staphylococcus species based on different partial gap, 16S rRNA, hsp60, rpoB, sodA, and tuf gene sequences. *J. Clin. Microbiol.* **46**, 1019–1025 (2008).
23. Suzuki, H., Lefébure, T., Bitar, P. P. & Stanhope, M. J. Comparative genomic analysis of the genus Staphylococcus including Staphylococcus aureus and its newly described sister species Staphylococcus simiae. *BMC Genomics* **13**, 38 (2012).
24. Trülsch, K. *et al.* “Staphylococcus pettenkoferi,” a novel staphylococcal species isolated from clinical specimens. *Diagn. Microbiol. Infect. Dis.* **43**, 175–182 (2002).
25. Huerta-Cepas, J., Serra, F. & Bork, P. ETE 3: Reconstruction, Analysis, and Visualization of Phylogenomic Data. *Mol. Biol. Evol.* **33**, 1635–1638 (2016).
26. Yu, G. *Data Integration, Manipulation and Visualization of Phylogenetic Trees. Data Integration, Manipulation and Visualization of Phylogenetic Trees* (Chapman and Hall, New York, 2022). doi:<https://doi.org/10.1201/9781003279242>.
27. Yu, G., Smith, D. K., Zhu, H., Guan, Y. & Lam, T. T. Y. ggtree: an r package for visualization and annotation of phylogenetic trees with their covariates and other associated data. *Methods Ecol. Evol.* **8**, 28–36 (2017).

28. Steinegger, M. & Söding, J. MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. *Nature Biotechnology* **2017 35:11 35**, 1026–1028 (2017).
29. Li, W. & Godzik, A. Cd-hit: a fast program for clustering and comparing large sets of protein or nucleotide sequences. *Bioinformatics* **22**, 1658–1659 (2006).
30. Fu, L., Niu, B., Zhu, Z., Wu, S. & Li, W. CD-HIT: accelerated for clustering the next-generation sequencing data. *Bioinformatics* **28**, 3150–3152 (2012).